

Web Browser Simulation System

Data Structures & Algorithms

Group Members:

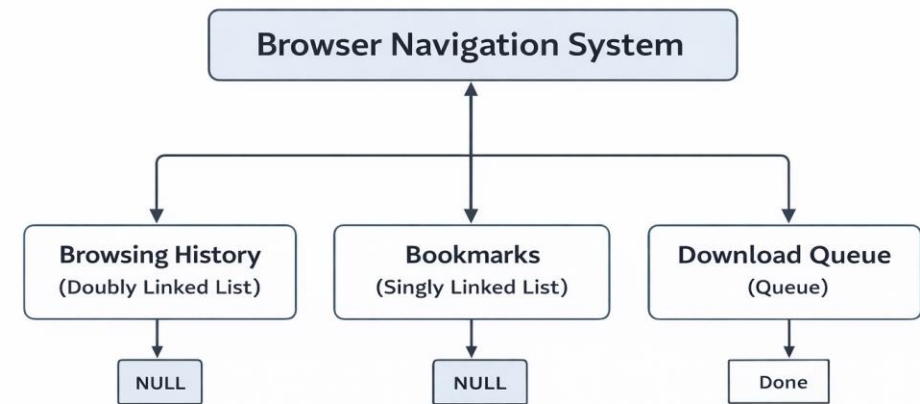
Abdul Rehman Ali
Syed Ali Abbas
Muhammad Ayaz
Ahmad Hassan
Maaz Qaisar

Introduction

Project Overview

We created a simulation of the core engine that powers a web browser. Using C++, we built the logic that handles navigation, memory, and data management.

Instead of a graphical interface with buttons, we focused on the **backend logic**: managing browsing history, saving bookmarks instantly, and handling file downloads in the correct order.



Project Objectives

- ✓ **Practical Application:** Understand how Data Structures work in real-world software.
- ✓ **Memory Management:** Implement efficient memory usage using C++ pointers.
- ✓ **Problem Solving:** Solve logic problems like "Back/Forward" navigation using Doubly Linked Lists.
- ✓ **Task Scheduling:** Manage download tasks efficiently using Queues (FIFO).
- ✓ **Code Quality:** Write clean, structured, and modular Object-Oriented code.

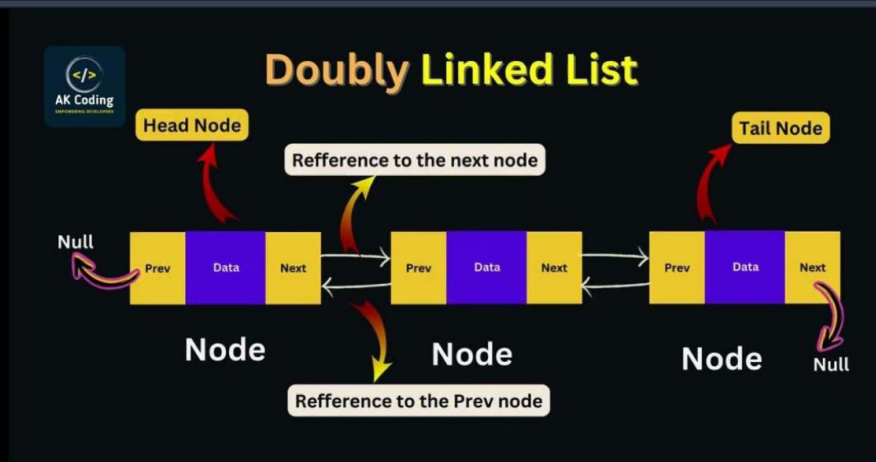
Page Structure

The "Page" Node

We use a **struct** to represent a webpage. It is a node in a Doubly Linked List.

We need pointers to both the **previous** page (for the Back button) and the **next** page (for the Forward button).

```
struct Page {  
    string url;  
    string title;  
    Page* prev;  
    Page* next;  
  
    Page(string u, string t) {  
        url = u;  
        title = t;  
        prev = nullptr;  
        next = nullptr;  
    }  
};
```



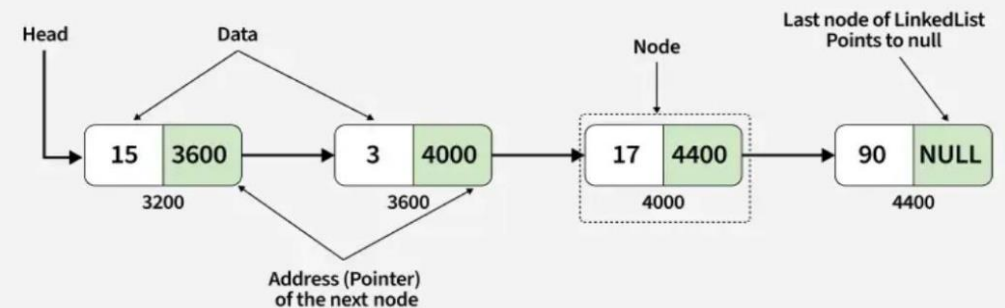
Bookmark Structure

The "Bookmark" Node

Bookmarks are stored in a simple list. We use a **Singly Linked List** because we only need to scan them from start to finish.

It holds the URL and Title, plus a pointer to the next bookmark.

```
struct Bookmark {  
    string url;  
    string title;  
    Bookmark* next;  
  
    Bookmark(string u, string t) {  
        url = u;  
        title = t;  
        next = nullptr;  
    }  
};
```



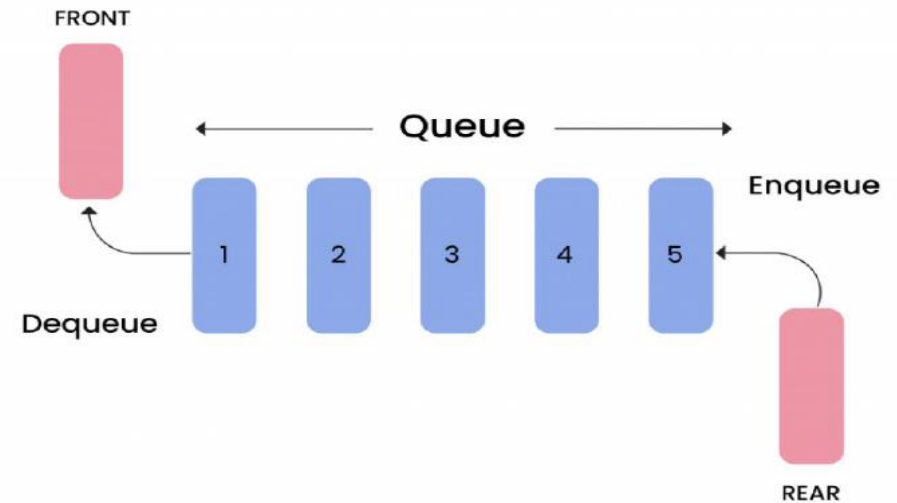
Download Structure

The Download Item

This structure is very simple. It just holds the name of the file being downloaded.

These items are stored in a **Queue**, ensuring fairness (First-In, First-Out).

```
struct Download {  
    string filename;  
    Download(string f) {  
        filename = f;  
    }  
};
```



Browser Class Overview

The Browser class is the central controller. It combines all our data structures into one functional system.

- ✓ **History Engine:** Uses Doubly Linked List logic.
- ✓ **Bookmark Manager:** Uses Singly Linked List logic.
- ✓ **Download Manager:** Uses Queue logic (STL).



Variables & Constructor

Pointers & Logic

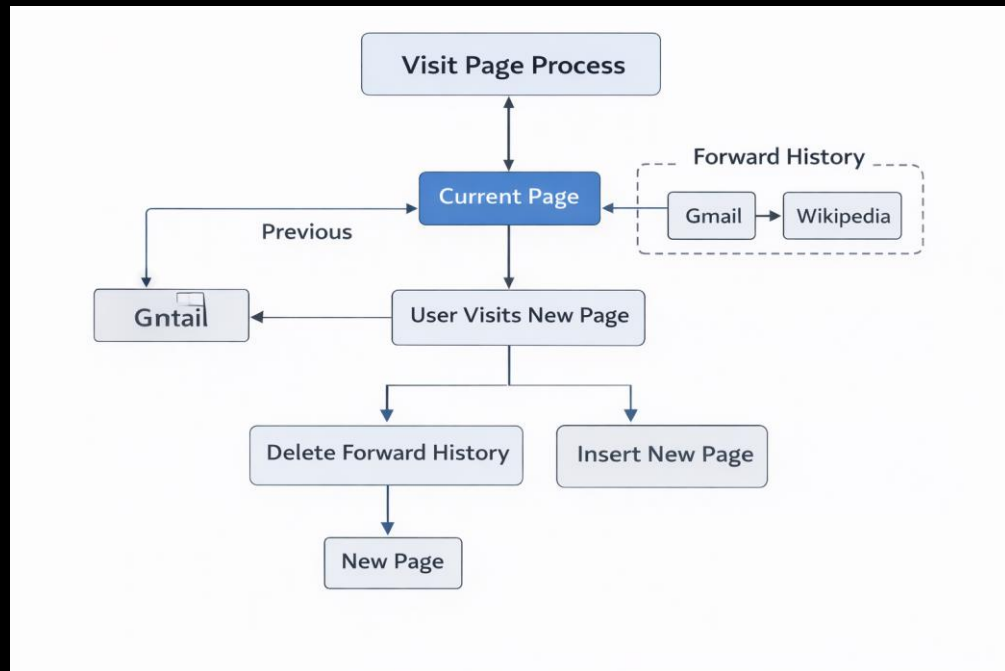
We initialize our system with nullptr to avoid memory errors.

We also set a default homepage immediately upon starting.

```
class Browser {  
private:  
    Page* head;  
    Page* tail;  
    Page* current;  
    Bookmark* bookmarksHead;  
    queue<Download> downloadQueue;  
  
public:  
    Browser() {  
        head = tail = current = nullptr;  
        bookmarksHead = nullptr;  
        visitPage("www.google.com", "Google Home");  
    }  
};
```


Function: Visit Page

When visiting a new page, we must cut off any "forward" history (just like Chrome or Edge). Then we link the new page.



```
void visitPage(const string& url, const string& title) {
    Page* newPage = new Page(url, title);
    if (!head) {
        head = tail = current = newPage;
        cout << "Visited: " << title << endl;
        return;
    }
    // Remove forward chain logic here ...
    current->next = newPage;
    newPage->prev = current;
    current = newPage;
    tail = newPage;
    cout << "Visited: " << title << endl;
}
```

Functions: Back & Forward

Go Back

Checks if a previous page exists, then moves the pointer back.

```
void goBack() {  
    if (current->prev == nullptr) {  
        cout << "No previous page.\n";  
        return;  
    }  
    current = current->prev;  
    cout << "Current: " << current->title << endl;  
}
```

Go Forward

Checks if a next page exists, then moves the pointer forward.

```
void goForward() {  
    if (current->next == nullptr) {  
        cout << "No forward page.\n";  
        return;  
    }  
    current = current->next;  
    cout << "Current: " << current->title << endl;  
}
```

Functions: View & Search History

View History

```
void viewHistory() {
    Page* temp = head;
    while (temp) {
        if (temp == current)
            cout << " > " << temp->title << " [CURRENT]\n";
        else
            cout << "    " << temp->title << endl;
        temp = temp->next;
    }
}
```

Search History

```
void searchHistory(const string& title) {
    Page* temp = head;
    while (temp && temp->title != title)
        temp = temp->next;

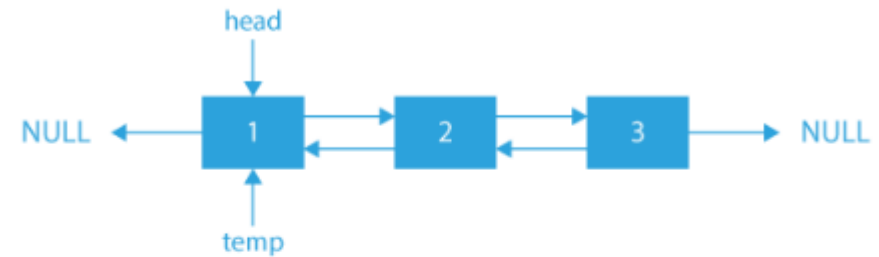
    if (!temp) {
        cout << "Page NOT found.\n";
        return;
    }
    cout << "Page FOUND: " << temp->title << endl;
}
```

Functions: Delete & Clear

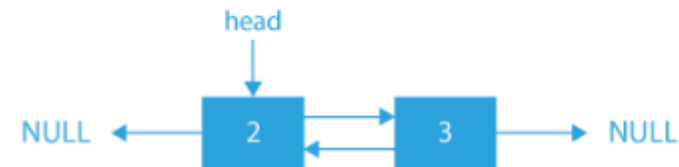
```
void deletePage(const string& title) {
    Page* temp = head;
    while (temp && temp->title != title) temp = temp->next;
    if (!temp) return;

    if (temp == current) { /* move current safely */ }
    if (temp->prev) temp->prev->next = temp->next;
    if (temp->next) temp->next->prev = temp->prev;
    // Handle head/tail updates ...
    delete temp;
}

void clearHistory() {
    Page* temp = head;
    while (temp) {
        Page* d = temp;
        temp = temp->next;
        delete d;
    }
    head = tail = current = nullptr;
}
```



We have to delete node to which temp is pointing
make, head = head → next
and then delete (temp)

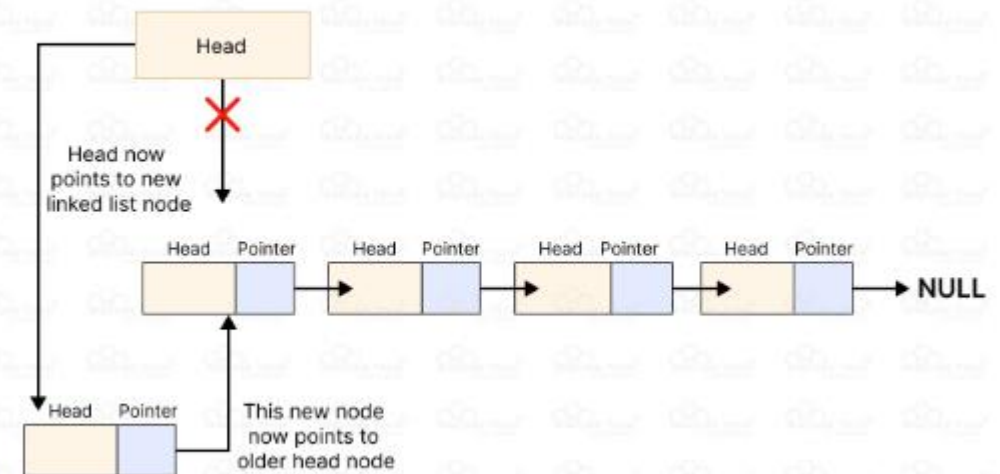


Functions: Bookmarks

```
void addBookmark(const string& url, const string& title) {  
    Bookmark* bm = new Bookmark(url, title);  
    bm->next = bookmarksHead;  
    bookmarksHead = bm;  
    cout << "Bookmark added.\n";  
}
```

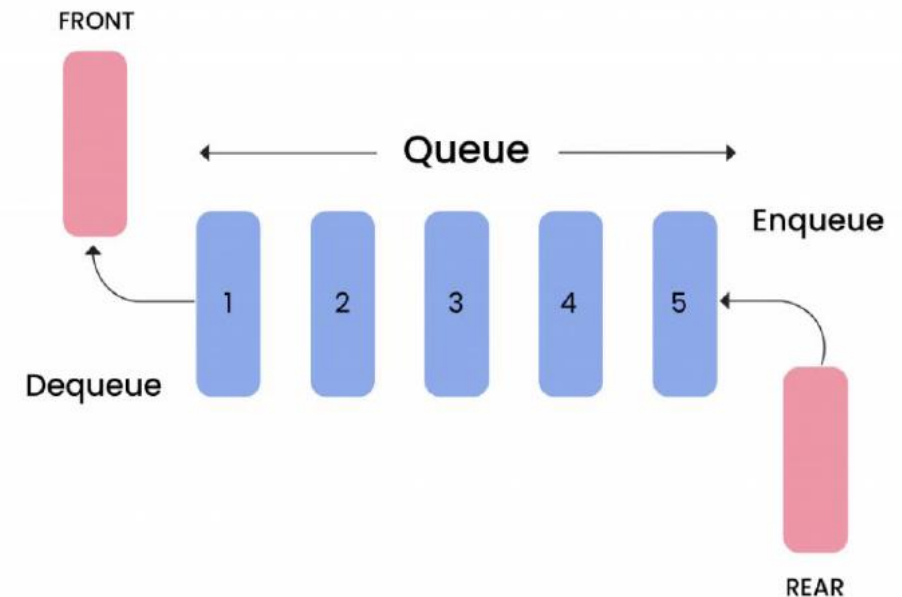
```
void deleteBookmark(const string& title) {  
    Bookmark* temp = bookmarksHead;  
    Bookmark* prev = nullptr;  
    while (temp && temp->title != title) {  
        prev = temp;  
        temp = temp->next;  
    }  
    if (!temp) return;  
    if (!prev) bookmarksHead = temp->next;  
    else prev->next = temp->next;  
    delete temp;  
}
```

Insertion at Beginning



Functions: Download System

```
void addDownload(const string& file) {  
    downloadQueue.push(Download(file));  
    cout << "Download added.\n";  
}  
  
void processDownload() {  
    if (downloadQueue.empty()) return;  
    Download d = downloadQueue.front();  
    downloadQueue.pop();  
    cout << "Downloaded: " << d.filename << endl;  
}  
  
void viewDownloads() {  
    queue<Download> temp = downloadQueue;  
    while (!temp.empty()) {  
        cout << " - " << temp.front().filename << endl;  
        temp.pop();  
    }  
}
```



Main Menu & Display

```
=====
Current Page: Google Home (www.google.com)
=====
1. Visit Page
2. Back
3. Forward
4. View History
5. Delete Page
6. Clear History
7. Add Bookmark
8. Delete Bookmark
9. View Bookmarks
10. Add Download
11. Process Download
12. View Downloads
13. Search Page by Title
14. Exit
Select: 1
URL: www.wikipedia.com
Title: WikiPedia
Visited: WikiPedia
Press any key to continue . . . █
```

Key Algorithms & Complexity

Operation	Time Complexity	Description
Visiting a Page	$O(1)$	Constant time insertion at current pointer.
Back / Forward	$O(1)$	Immediate pointer movement (prev/next).
Search History	$O(N)$	Linear search required to find a title.
Clear History	$O(N)$	Traversing and deleting all nodes to free memory.

Summary

We built a functional simulation of a web browser using C++.

By combining **Linked Lists** for history/bookmarks and **Queues** for downloads, we created a system that mimics real world behavior.

| This project proves that choosing the right data structure is the key to efficient software design.

Conclusion

Final Thoughts

In this project, we successfully modeled the core engine of a web browser.

We utilized **Doubly Linked Lists** to enable flexible navigation history, **Singly Linked Lists** for efficient bookmark storage, and **Queues** for orderly download management.

This simulation demonstrates the practical power of Data Structures in solving real-world software engineering problems.



Thank You!

Questions?